

Assignment 01

Neural Networks and Deep Learning

SE4022: Machine Learning

2026 July

Student Name	IT number
R.M.C.S.Rathnayaka	IT23231528

BSc. (Hons) in Computer Science

Department of Computer Science

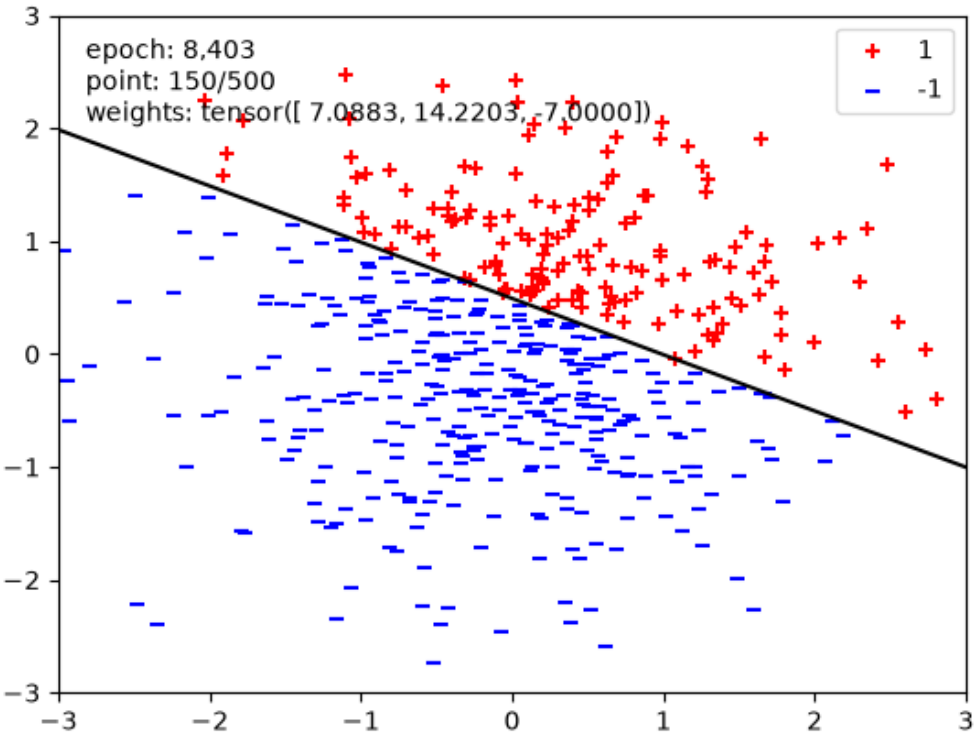
Sri Lanka Institute of Information Technology

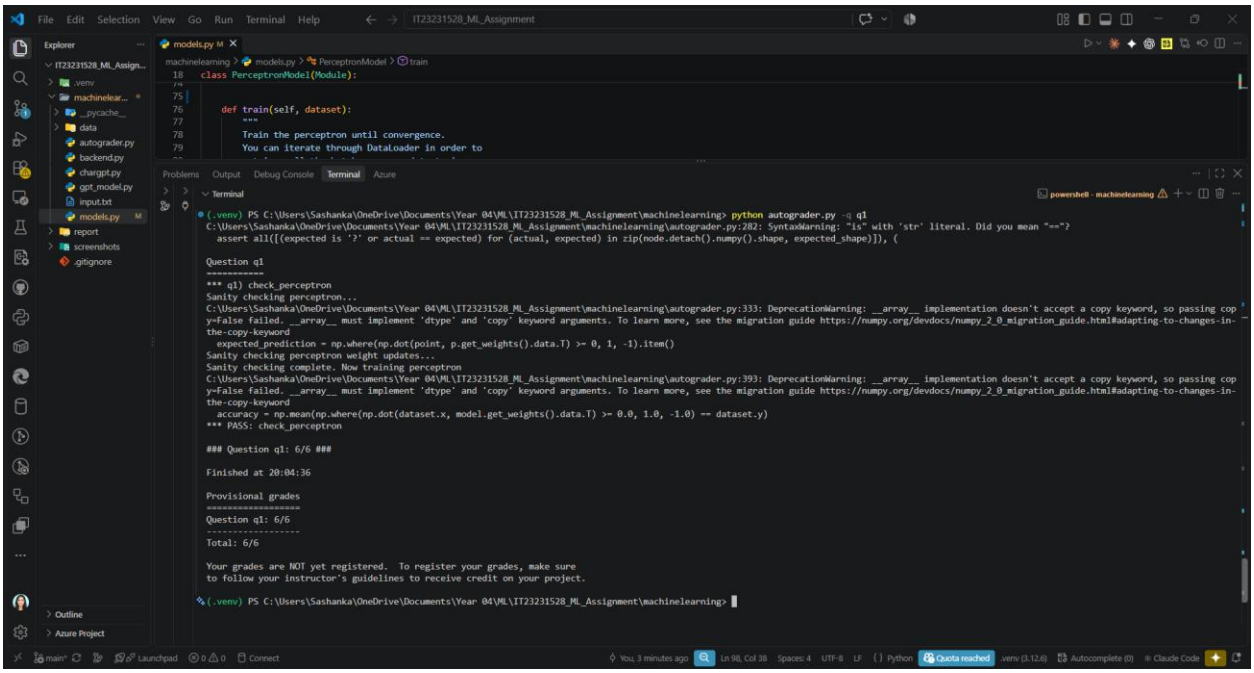
Git Repository link:

https://github.com/IT23231528chamudi/IT23231528_ML_Assignment.git

Task 1: The Perceptron

1. Autograder Output





2. Implementation

```python
class PerceptronModel(Module):
    def __init__(self, dimensions):
        super(PerceptronModel, self).__init__()

        # Initialize the weight parameter with ones
        self.w = Parameter(ones(1, dimensions))

    def get_weights(self):
        return self.w   # Return the current weight parameter

    def run(self, x):

        # Calculate the perceptron score by multiplying weights with input features
        return torch.sum(self.w * x)

    def get_prediction(self, x):
        score = self.run(x)  # Calculate the score for the given input

        # Predict positive class if the score is non negative
        if score.item() >= 0:
            return 1
        else:
            return -1
```

```python
    def train(self, dataset):

        # Disable automatic gradient calculation for perceptron updates
        with no_grad():
            # Create a data loader that processes one sample at a time
            dataloader = DataLoader(dataset, batch_size=1, shuffle=True)

        # Continue training until the model makes no mistakes
        while True:
            mistakes = 0
            for data in dataloader:
                    x = data['x']
                    y = data['label']

                    prediction = self.get_prediction(x)

                    if prediction != y.item():
                        # Update the weights using the true label and input features
                        with torch.no_grad():
                            self.w.add_(y * x)
                        mistakes += 1

            if mistakes == 0:
                    break
```

The following functions were implemented in **PerceptronModel**:

- **__init__(self, dimensions):** Initializes the weight parameter as a PyTorch Parameter containing ones, with dimensions matching the input feature size
- **get_weights(self):** Returns the current weight parameter
- **run(self, x):** Calculates the perceptron score by multiplying the weights by the input features and summing the results
- **get_prediction(self, x):** Predicts the positive class (+1) when the score is non-negative and the negative class (-1) otherwise
- **train(self, dataset):** Trains the model using the perceptron learning rule until a complete pass through the dataset produces no misclassifications

The weight update is performed using the following statement: **self.w.add_(y * x)**

This updates the weights using the true label and input features whenever a prediction is incorrect

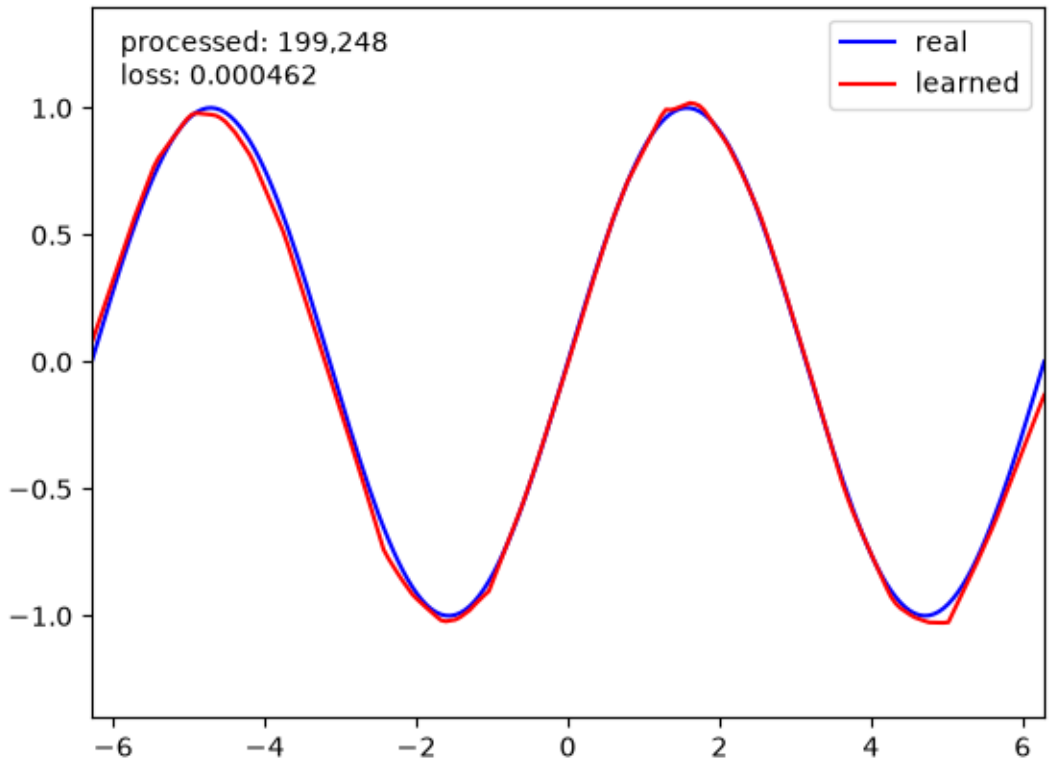
3. Explanation of the implementation

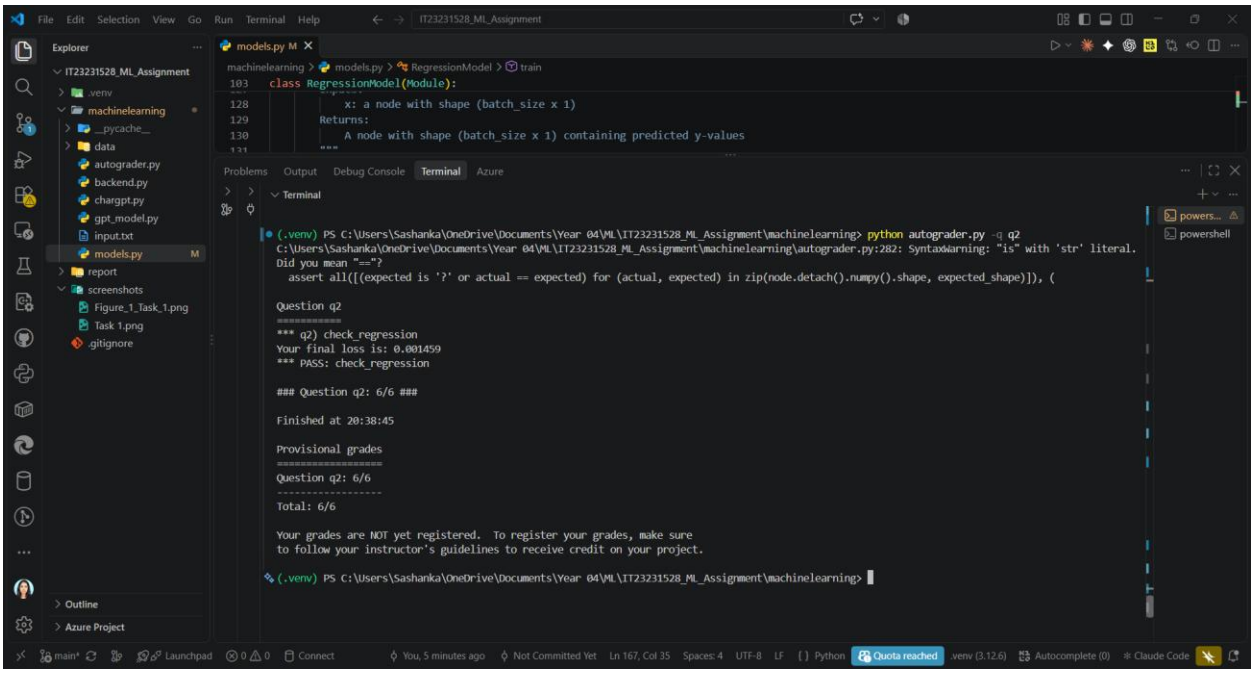
The Perceptron is a binary classification model that predicts whether an input belongs to class +1 or class -1. The model initializes its weight vector with ones and calculates a score by summing the element wise products of the weights and input features. The sign of this score determines the predicted class

During training, the dataset is processed one sample at a time using a DataLoader with a batch size of 1. Whenever the predicted label differs from the true label, the weight vector is updated using the perceptron learning rule. Automatic gradient calculation is disabled because the weights are updated directly rather than through backpropagation. Training continues until an entire pass through the dataset produces no mistakes indicating that the model has converged on the training data

Task 2: Non-Linear Regression

1. Autograder Output





2. Implementation

```python
class RegressionModel(Module):
    def __init__(self):
        # Initialize the parent PyTorch module
        super().__init__()

        # Initialize the first hidden layer with one input feature and 128 neurons
        self.layer1 = Linear(1, 128)

        # Initialize the second hidden layer with 128 neurons
        self.layer2 = Linear(128, 128)

        # Initialize the output layer to predict a single value
        self.output_layer = Linear(128, 1)

    def run(self, x):
        x = relu(self.layer1(x))   # Pass the input through the first hidden layer
        x = relu(self.layer2(x))   # Pass the features through the second hidden layer
        return self.output_layer(x)      # Calculate and return the predicted output


    def forward(self, x):

        # Run the model to obtain predictions
        return self.run(x)

    def get_loss(self, x, y):

        # Calculate the predicted values for the input batch
        prediction = self.run(x)

        # Calculate the mean squared error between predictions and true values
        return mse_loss(prediction, y)
```

```python
    def train(self, dataset):
        # Define the number of samples in each training batch
        batch_size = 64

        # Set the learning rate for the optimizer
        learning_rate = 0.01

        # Set the number of training epochs
        epochs = 1000

        # Create a data loader to load shuffled mini-batches
        dataloader = DataLoader(
            dataset,
            batch_size=batch_size,
            shuffle=True
        )

        # Initialize the Adam optimizer with the model parameters
        optimizer = optim.Adam(
            self.parameters(),
            lr=learning_rate
        )

        # Iterate through the dataset for the specified number of epochs
        for epoch in range(epochs):

            # Process each mini-batch in the dataset
            for batch in dataloader:
                x = batch['x']
                y = batch['label']

                optimizer.zero_grad()
                loss = self.get_loss(x, y)
                loss.backward()
                optimizer.step()  # Update the model parameters using the optimizer
```

The **RegressionModel** was implemented as a feedforward neural network using PyTorch. The main functions and their purposes are as follows:

- **__init__(self):** Defines two hidden layers containing 128 neurons each and an output layer containing one neuron
- **run(self, x):** Passes the input through the two hidden layers, applying the ReLU activation function after each layer, and produces the predicted output
- **forward(self, x):** Calls run() to obtain the model predictions
- **get_loss(self, x, y):** Calculates the mean squared error (MSE) between the predicted and actual target values
- **train(self, dataset):** Trains the network using mini batches, the Adam optimizer, and backpropagation

The following code illustrates the forward computation:

- x = relu(self.layer1(x))
- x = relu(self.layer2(x))
- return self.output_layer(x)

The training process uses a batch size of 64, a learning rate of 0.01, and 1,000 epochs

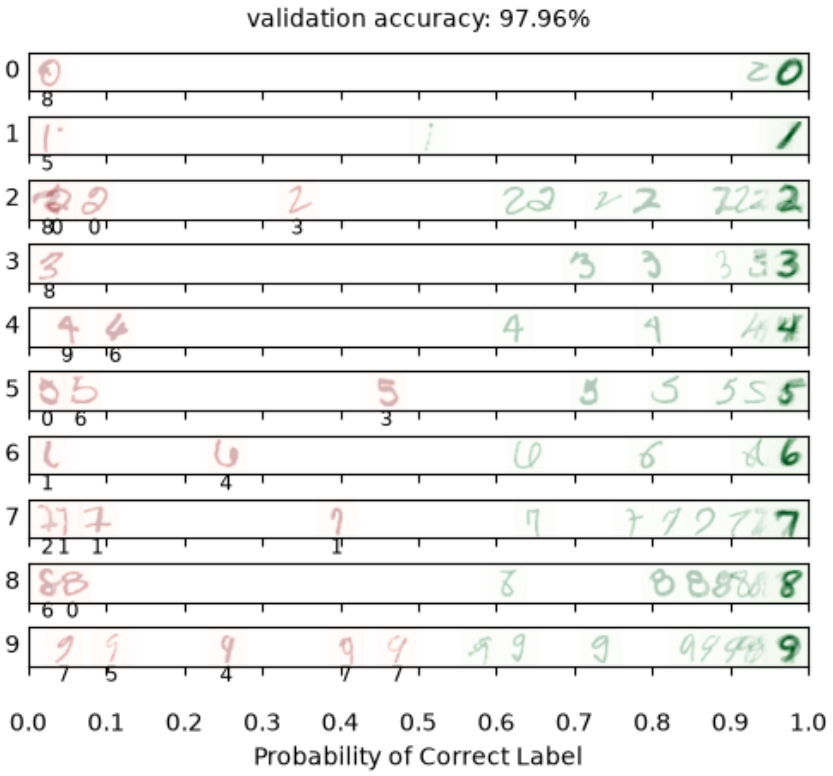
3. Explanation of the implementation

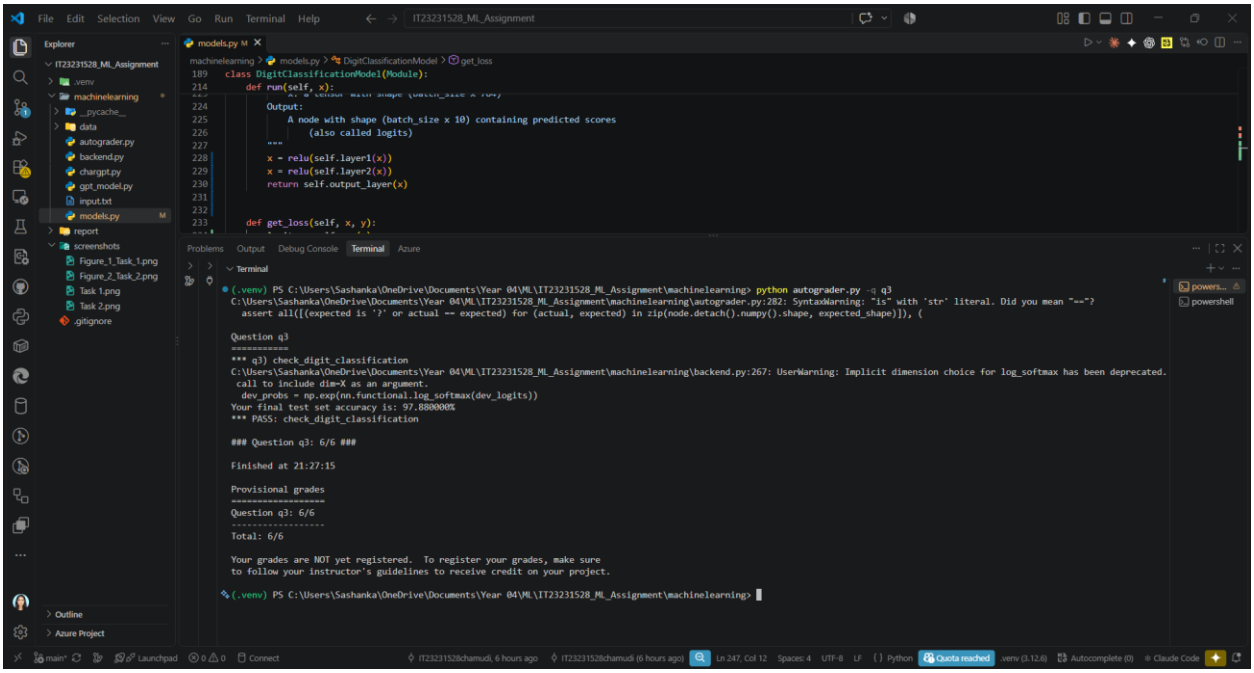
The model approximates a function that maps real valued inputs to real valued outputs, with the aim of approximating the sine function over the interval (-2π, 2π). Two hidden layers with 128 neurons each provide the network with the capacity to learn nonlinear relationships. The ReLU activation function is applied after each hidden layer, while the final linear layer produces a single continuous output

Mean squared error is used as the loss function because the task involves predicting continuous values. The model is trained using the Adam optimizer with a learning rate of 0.01 and a batch size of 64. During each training iteration, the gradients are cleared, the loss is calculated, backpropagation computes the gradients, and the optimizer updates the model parameters. Training runs for 1,000 epochs to allow the network to improve its approximation of the target function

Task 3: Digit Classification

1. Autograder Output





2. Implementation

```python
class DigitClassificationModel(Module):

    def __init__(self):
        # Initialize your model parameters here
        super().__init__()
        input_size = 28 * 28
        output_size = 10

        self.layer1 = Linear(input_size, 256)
        self.layer2 = Linear(256, 128)
        self.output_layer = Linear(128, output_size)


    def run(self, x):

        # Pass the input images through the first hidden layer
        x = relu(self.layer1(x))

        # Pass the extracted features through the second hidden layer
        x = relu(self.layer2(x))
        return self.output_layer(x)


    def get_loss(self, x, y):

        # Calculate the output scores for the input images
        logits = self.run(x)
        return cross_entropy(logits, y)
```

```python
    def train(self, dataset):
        batch_size = 64
        learning_rate = 0.001
        epochs = 15

        dataloader = DataLoader(
            dataset,
            batch_size=batch_size,
            shuffle=True
        )

        # Initialize the Adam optimizer with the model parameters
        optimizer = optim.Adam(
            self.parameters(),
            lr=learning_rate
        )

        # Iterate through the dataset for the specified number of epochs
        for epoch in range(epochs):
            for batch in dataloader:
                x = batch['x']
                y = batch['label']

                optimizer.zero_grad()
                loss = self.get_loss(x, y)
                loss.backward()
                optimizer.step()
```

The **DigitClassificationModel** was implemented as a feedforward neural network using PyTorch to classify handwritten digits from the MNIST dataset. Each input image is represented as a flattened vector containing 784 pixel values, corresponding to a (28 \times 28) grayscale image

The main functions are:

- **__init__(self):** Initializes two hidden layers containing 256 and 128 neurons, respectively, and an output layer with 10 neurons, representing the digit classes from 0 to 9
- **run(self, x):** Passes the input through the hidden layers using ReLU activation and returns the output scores for the 10 digit classes
- **get_loss(self, x, y):** Calculates the cross-entropy loss between the predicted scores and the true digit labels
- **train(self, dataset):** Trains the model using mini batches, the Adam optimizer, and backpropagation

The forward computation is implemented as follows:

- x = relu(self.layer1(x))
- x = relu(self.layer2(x))
- return self.output_layer(x)

The model uses a batch size of 64, a learning rate of 0.001, and 15 training epochs

3. Explanation of the implementation

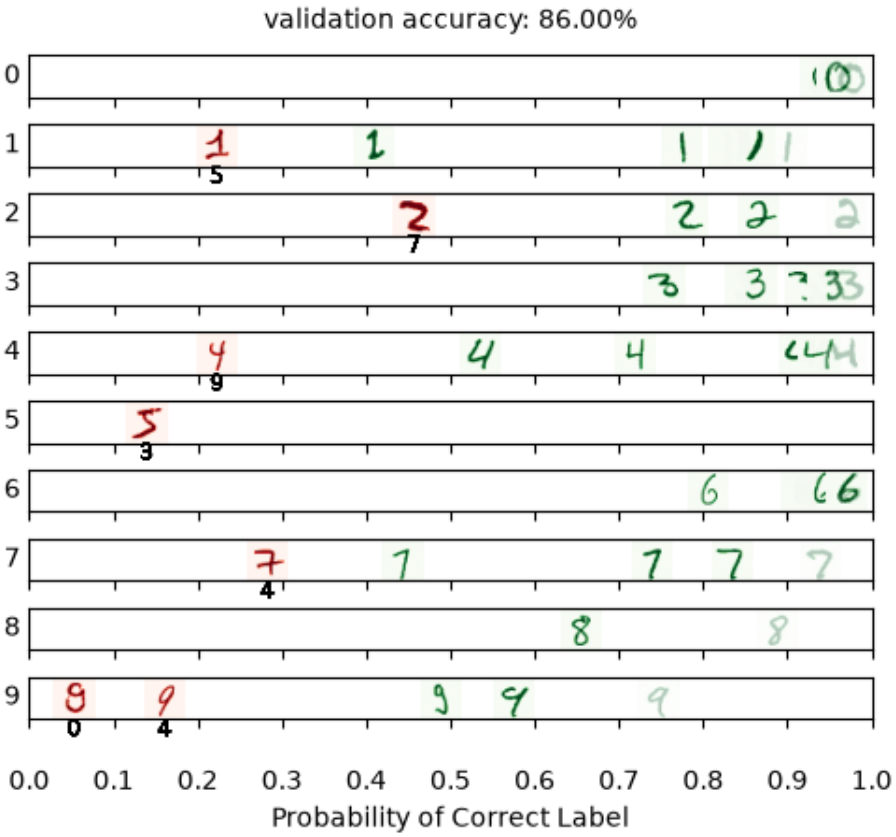
The model uses a feedforward neural network to learn patterns in handwritten digit images. The input layer accepts 784 features, corresponding to the flattened pixel values of each image. The first hidden layer contains 256 neurons, and the second contains 128 neurons. ReLU activation functions introduce nonlinearity, allowing the network to learn more complex visual patterns. The final layer contains 10 output neurons, one for each digit class

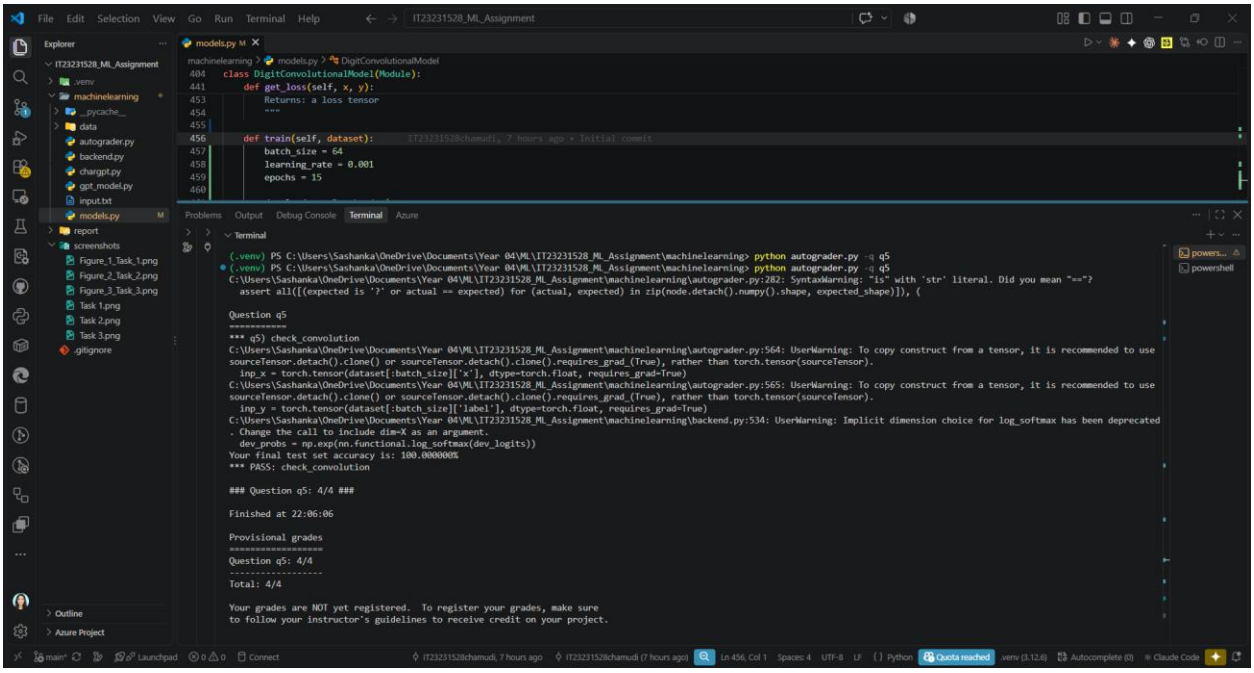
Cross-entropy loss is used to measure the difference between the predicted class scores and the correct labels. The Adam optimizer updates the model parameters during training. A batch size of 64 is used to process groups of training examples, while the learning rate of 0.001 controls the size of the optimizer's updates. Training is performed for 15 epochs to allow the model to learn useful features for digit classification

During each training iteration, the gradients are cleared, the loss is calculated, backpropagation computes the gradients, and the optimizer updates the model parameters

Bonus Task: Convolutional Neural Networks

1. Autograder Output





2. Implementation

```
312  def Convolve(input: tensor, weight: tensor):
313
314      input_height, input_width = input.shape
315      weight_height, weight_width = weight.shape
316
317      output_height = input_height - weight_height + 1
318      output_width = input_width - weight_width + 1
319
320      output_rows = []
321
322      # Slide the filter vertically across the input image
323      for i in range(output_height):
324          output_row = []
325
326          # Slide the filter horizontally across the input image
327          for j in range(output_width):
328
329              # Extract the current region of the input image covered by the filter
330              region = input[
331                  i:i + weight_height,
332                  j:j + weight_width]
333
334              value = tensordot(region, weight, dims=2)
335              output_row.append(value)
336
337
338          # Stack the values to form the current output row
339          output_rows.append(stack(output_row))
340
341      # Stack all output rows to create the final convolved output
342      return stack(output_rows)
```

Convolve Function:

The **Convolve()** function implements a two-dimensional convolution operation without using a built-in PyTorch convolution function. It receives an input image and a filter weight matrix as arguments

The function first obtains the dimensions of the input image and filter, then calculates the output dimensions. Nested loops slide the filter across the input image, extracting a region of the same size as the filter at each position. The tensordot() function calculates the sum of the element-wise products between the extracted region and the filter weights. The resulting values are collected into rows and combined using stack() to produce the final convolved output

The following code performs the main convolution calculation:

```
region = input[
    i:i + weight_height,
    j:j + weight_width]

value = tensordot(region, weight, dims=2)
output_row.append(value)
```

This approach implements the convolution operation using tensor indexing, tensordot(), and stack()

Convolutional Model:

```python
class DigitConvolutionalModel(Module):

    def __init__(self):
        # Initialize your model parameters here
        super().__init__()
        output_size = 10 # classes representing digits 0 through 9

        self.convolution_weights = Parameter(ones((3, 3)))
        self.layer1 = Linear(26 * 26, 64)
        self.output_layer = Linear(64, 10)

    def run(self, x):

        # Pass the input through the forward method
        return self.forward(x)

    def forward(self, x):

        x = x.reshape(len(x), 28, 28)
        x = stack(list(map(lambda sample: Convolve(sample, self.convolution_weights), x)))
        x = x.flatten(start_dim=1)
        x = relu(self.layer1(x))
        return self.output_layer(x)


    def get_loss(self, x, y):

        # Calculate the output scores for the input images
        logits = self.run(x)

        return cross_entropy(self.run(x), y)
```

```python
    def train(self, dataset):
        batch_size = 64
        learning_rate = 0.001
        epochs = 15

        dataloader = DataLoader(
            dataset,
            batch_size=batch_size,
            shuffle=True
        )

        optimizer = optim.Adam(
            self.parameters(),
            lr=learning_rate
        )

        for epoch in range(epochs):
            for batch in dataloader:
                x = batch['x']
                y = batch['label']

                optimizer.zero_grad()
                loss = self.get_loss(x, y)
                loss.backward()
                optimizer.step()
```

The **DigitConvolutionalModel** uses the custom Convolve() function to extract features from handwritten digit images. Its main components are:

- **__init__():** Initializes a (3 \times 3) convolution filter and two fully connected layers. The first fully connected layer contains 64 neurons, while the output layer contains 10 neurons for digit classification
- **run():** Calls the forward() method to obtain predictions
- **forward():** Reshapes each flattened input into a (28 \times 28) image, applies the custom convolution function, flattens the resulting feature maps, applies a fully connected layer with ReLU activation, and produces scores for the 10 digit classes
- **get_loss():** Calculates the cross-entropy loss using the predicted scores and the true labels
- **train():** Defines a mini-batch training procedure using the Adam optimizer and backpropagation

The following code illustrates the feature extraction and classification process:

```
def forward(self, x):

    x = x.reshape(len(x), 28, 28)
    x = stack(list(map(lambda sample: Convolve(sample, self.convolution_weights), x)))
    x = x.flatten(start_dim=1)
    x = relu(self.layer1(x))
    return self.output_layer(x)
```

3. Explanation of the implementation

Convolve Function:

The custom convolution function applies a (3 \times 3) filter to an input image by moving the filter across valid positions without padding. At each position, the corresponding image region is multiplied element by element with the filter weights, and the products are summed to produce one output value. The output dimensions are calculated using the input and filter dimensions. This implementation follows the assignment requirement to perform convolution without calling a built-in PyTorch convolution operation

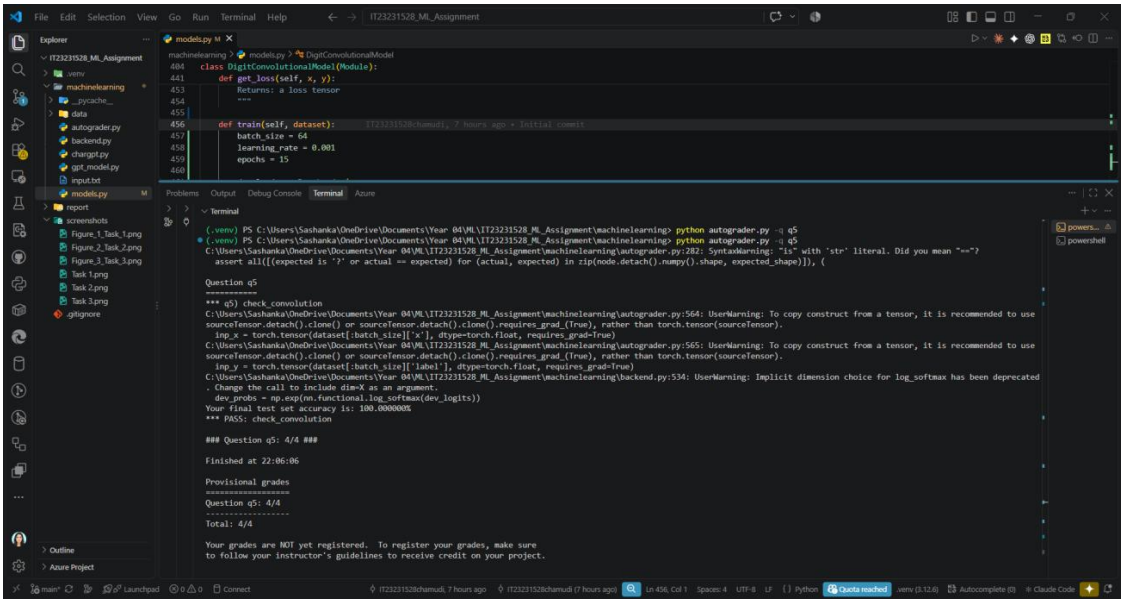
Convolutional Model:

The convolutional model processes each (28 \times 28) image using the custom convolution function and a (3 \times 3) filter. With no padding and a stride of one, the resulting feature map has dimensions of (26 \times 26). The feature map is flattened into a vector of 676 values and passed to a fully connected layer with 64 neurons. ReLU activation introduces nonlinearity, and the final layer produces 10 scores corresponding to the possible digit classes

Cross entropy loss is used to measure classification error. The provided code also defines training with a batch size of 64, a learning rate of 0.001, 15 epochs, and the Adam optimizer. However, the class description indicates that the model is intended to use preset weights, so the training method is not necessarily required for the bonus task. The main objective is to implement the custom convolution operation correctly so that the model can use the provided weights to classify handwritten digits

Task 4: Language Identification Model

1. Autograder Output



2. Implementation

```python
class LanguageIDModel(Module):
    def __init__(self):
        # Our dataset contains words from five different languages, and the
        # combined alphabets of the five languages contain a total of 47 unique
        # characters.
        # You can refer to self.num_chars or len(self.languages) in your code
        self.num_chars = 47
        self.languages = ["English", "Spanish", "Finnish", "Dutch", "Polish"]
        super(LanguageIDModel, self).__init__()
        self.hidden_size = 128

        self.char_layer = Linear(self.num_chars, self.hidden_size)
        self.hidden_layer = Linear(self.hidden_size, self.hidden_size)
        self.output_layer = Linear(self.hidden_size, len(self.languages))

    def run(self, xs):

        # Get the number of words in the current batch
        batch_size = xs[0].shape[0]

        # Initialize the hidden state with zeros for each word
        hidden = xs[0].new_zeros((batch_size, self.hidden_size))

        # Process each character in the input word sequence
        for x in xs:

            # Combine the current character representation with the previous hidden state
            hidden = relu(self.char_layer(x) + self.hidden_layer(hidden))

        return self.output_layer(hidden)
```

```
274     def get_loss(self, xs, y):
275         logits = self.run(xs)
276         return cross_entropy(logits, y)
277 
278     def train(self, dataset):
279         batch_size = 64
280         learning_rate = 0.001
281         epochs = 25
282 
283         dataloader = DataLoader(dataset,batch_size=batch_size,shuffle=True)
284 
285         optimizer = optim.Adam(self.parameters(),lr=learning_rate)
286 
287         for epoch in range(epochs):
288             for batch in dataloader:
289                 x = movedim(batch['x'], 1, 0)
290                 y = batch['label']
291 
292                 optimizer.zero_grad()
293                 loss = self.get_loss(x, y)
294                 loss.backward()
295                 optimizer.step()
```

The **LanguageIDModel** was implemented as a recurrent neural network (RNN) using PyTorch to identify the language of a word based on its character sequence. The dataset contains five languages: English, Spanish, Finnish, Dutch, and Polish, with a combined alphabet of 47 unique characters

The main functions are:

- **__init__(self):** Initializes the character processing layer, recurrent hidden layer, and output layer. The hidden state contains 128 features, and the output layer produces five language scores
- **run(self, xs):** Processes the characters sequentially, updates the hidden state using the current character and previous hidden state, and generates scores for the five languages
- **get_loss(self, xs, y):** Calculates the cross entropy loss between the predicted language scores and the correct labels
- **train(self, dataset):** Trains the model using mini batches, the Adam optimizer, and backpropagation. The input dimensions are rearranged using movedim() so that the sequence length is the first dimension

The recurrent computation is implemented as follows:

```
# Initialize the hidden state with zeros for each word
hidden = xs[0].new_zeros((batch_size, self.hidden_size))

# Process each character in the input word sequence
for x in xs:

    # Combine the current character representation with the previous hidden state
    hidden = relu(self.char_layer(x) + self.hidden_layer(hidden))
```

The model uses a hidden size of 128, a batch size of 64, a learning rate of 0.001, and 25 training epoc

3. Explanation of the implementation

The model uses a recurrent neural network to classify words according to their character sequences. Each character is represented as a 47 dimensional one-hot vector. The character-processing layer transforms the current character into a hidden representation, while the recurrent hidden layer incorporates information

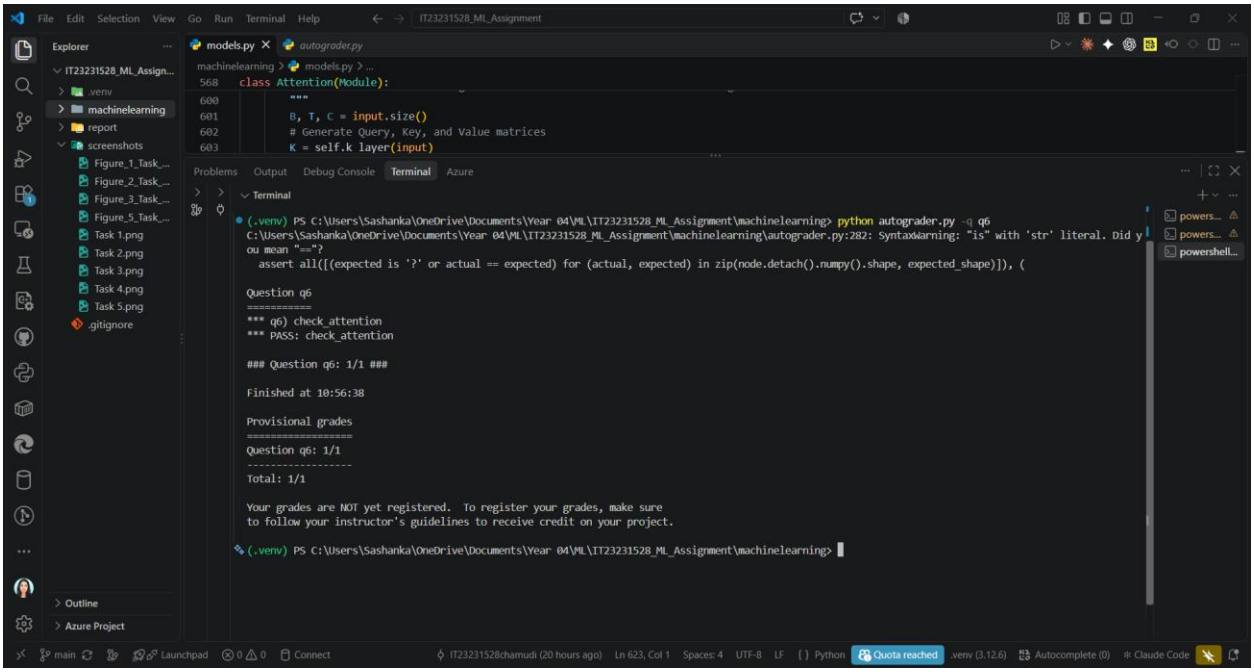
from the previous hidden state. The ReLU activation function is applied to the combined representations, allowing the model to learn nonlinear relationships between characters

After processing the complete word, the final hidden state is passed to the output layer, which generates five scores corresponding to the possible languages. Cross entropy loss is used to measure the difference between the predicted scores and the correct language labels

The model is trained using the Adam optimizer with a learning rate of 0.001 and a batch size of 64 for 25 epochs. The movedim() function rearranges each input batch from batch-first format to sequence first format, allowing the model to process characters in the intended order. These design choices allow the model to use information from the character sequence when predicting the language of a word

Task 6: Attention Mechanism

1. Autograder Output



2. Implementation

```
class Attention(Module):
    def __init__(self, layer_size, block_size):
        super().__init__()

        self.k_layer = Linear(layer_size, layer_size)
        self.q_layer = Linear(layer_size, layer_size)
        self.v_layer = Linear(layer_size,layer_size)

        #Masking part of attention layer
        self.register_buffer("mask", torch.tril(torch.ones(block_size, block_size))
                                     .view(1, 1, block_size, block_size))

        self.layer_size = layer_size


    def forward(self, input):

        B, T, C = input.size()
        # Generate Query, Key, and Value matrices
        K = self.k_layer(input)
        Q = self.q_layer(input)
        V = self.v_layer(input)

        # Calculate scaled attention scores: K @ Q^T
        scores = torch.matmul(K, torch.movedim(Q, 1, 2))
        scores = scores / (self.layer_size ** 0.5)

        # Apply causal masking
        scores = scores.masked_fill(
            self.mask[:, :, :T, :T] == 0,
            float('-inf'))[0]

        # Apply softmax along the last dimension
        attention_weights = softmax(scores, dim=-1)

        # Multiply attention weights by the Value matrix
        output = torch.matmul(attention_weights, V)

        return output
```

The Attention class implements a causal attention mechanism using PyTorch linear layers and tensor operations. The implementation contains the following components:

- **__init__(self, layer_size, block_size):** Initializes the key, query, and value linear layers. It also creates a lower-triangular causal mask using torch.tril() and registers the mask as a buffer
- **forward(self, input):** Generates the key, query, and value matrices, calculates scaled attention scores, applies the causal mask, computes attention weights using softmax, and combines the value vectors to produce the output

The main attention calculation is implemented as follows:

```
# Generate Query, Key, and Value matrices
K = self.k_layer(input)
Q = self.q_layer(input)
V = self.v_layer(input)

# Calculate scaled attention scores: K @ Q^T
scores = torch.matmul(K, torch.movedim(Q, 1, 2))
scores = scores / (self.layer_size ** 0.5)

# Apply causal masking
scores = scores.masked_fill(
    self.mask[:, :, :T, :T] == 0,
    float('-inf'))[0]

# Apply softmax along the last dimension      You, 11
attention_weights = softmax(scores, dim=-1)

# Multiply attention weights by the Value matrix
output = torch.matmul(attention_weights, V)
```

3. Explanation of the implementation

The attention mechanism calculates relationships between positions in an input sequence. The key, query, and value matrices are generated using separate linear layers. The key and query matrices are multiplied to calculate attention scores, which are scaled by the square root of the layer size to control their magnitude

A lower-triangular causal mask is applied to prevent each position from attending to future positions. Masked entries are assigned negative infinity, causing their attention weights to become zero after softmax. The softmax function is applied along the last dimension to obtain normalized attention weights. Finally, these weights are multiplied by the value matrix to produce the attention output

This implementation uses the provided layer definitions and masking structure while applying the required tensor operations directly. The causal mask ensures that the attention mechanism follows the sequence order without using information from future positions